Code Review: Legacy Data Ingestion (`legacy_ingest/`)

Cold-Chain Logistics AI-Assistant · CSV → MSSQL loader for `dbo.TBL_SC_FLEET_HIST_RAW`

Author	Jimit Naik
Date / version	2026-11-13 · v1.0
Code reviewed	`legacy_ingest/` (12 files, 929 lines) and its entry point `scripts/ingest_legacy_data.py`. Working tree on branch `feat/langgraph-secure-agent`, base commit `c2caa93`. The package is not committed yet.
Assessed for	Production: a scheduled job feeding the telemetry that the agent reads through `VW_ACTIVE_FLEET` (login `USR_FDE_RO`)
Reproduction tests	`tests/open_findings/test_legacy_open_findings.py`: 11 tests, all failing on the reviewed code

1. Decision Summary

Verdict: NO-GO for running the loader as a scheduled production job.
It becomes a GO once **F-01** (Critical) and **F-02** (High) are fixed and their reproduction tests pass in the main suite.

Why NO-GO. In normal operation, the first scheduled refresh replaces the live table with a new database object. Permissions, indexes and dependent views on `dbo.TBL_SC_FLEET_HIST_RAW` are lost (**F-01**), so the agent's read-only login can lose access to fleet telemetry while the loader reports `SUCCEEDED`. If the view is created `WITH SCHEMABINDING`, every refresh fails instead. Separately, if the database connection drops during a load, the error handler crashes. The real cause is hidden and the audit row stays `RUNNING` forever (**F-02**).

What is already safe. The data path is sound. A failed load never publishes partial data; the live table is either the old version or the complete new one. Invalid rows are quarantined, and the staging table is checked against the source before anything is published. The loader was verified end to end against the project's SQL Server 2022 instance: 32,065 rows loaded in 3.1 s, an unchanged re-run was skipped, and an upsert made 0 changes as expected. Running it manually today produces correct data. The blockers are about how it coexists with other database objects and how it behaves when things go wrong.

Severity	Count	Findings
CRITICAL	1	F-01 Refresh drops grants, indexes and views on the live table
HIGH	1	F-02 Failure handler crashes on a dead connection; audit stuck at RUNNING
MEDIUM	5	F-03 Deadlocks never retried · F-04 Upsert skips an emptied table · F-05 Audit hash can describe the wrong file · F-06 CSV missing a column exits 1, not 2 · F-07 Loader depends on `rag/`
LOW	4	F-08 Reject file hidden on failure · F-09 `--chunk-rows 0` ignored · F-10 Identifier check accepts a trailing newline · F-11 Source read three times; duplicate check slows quadratically

All 11 findings are **OPEN**. Each one has an automated test that fails on the reviewed code (Section 3 and Appendix A).

2. Scope & Method

IN SCOPE

`legacy_ingest/`: `config.py`, `schema.py`, `extract.py`, `transform.py`, `validate.py`, `db.py`, `audit.py`, `loader.py`, `cli.py`, `errors.py` and the backward-compatible wrapper `scripts/ingest_legacy_data.py`.

OUT OF SCOPE

The SOP retrieval platform (`rag/`) is covered by a separate review; it appears here only as a dependency in F-07. Also out of scope: SQL Server configuration and hardening, the creation of `VW_ACTIVE_FLEET` and `USR_FDE_RO`, infrastructure, and a dedicated security review.

HOW THE LOADER WORKS (CONTEXT FOR THE FINDINGS)

The loader reads the CSV in chunks and maps the 9 source columns to legacy names. It validates each row, sends rejected rows to `data/rejects/<run_id>.csv`, and bulk-inserts valid rows into `<table>__STG`. It then checks the staging table against the source (row count and the earliest and latest `TS_UTC`) and publishes in one of two modes:

- **refresh** (default): in one transaction, renames the live table to `__OLD`, renames the staging table to the live name, drops `__OLD` and renames the constraints.
- **upsert**: runs a `MERGE` into the live table on `TS_UTC`.

Each run records a row in `dbo.TBL_SC_INGEST_AUDIT`. The run is skipped if the CSV's SHA-256 matches the last successful load, and an application lock (`sp_getapplock`) stops two loads running at once.

METHOD

1. **Review.** One structured code-review pass over the package, looking for correctness, reliability, operability and performance defects.
2. **Proof.** Each finding was turned into a unit test that asserts the *correct* behaviour. All 11 tests fail on the reviewed code, each for the reason stated in its finding. The tests run offline against an in-memory SQL Server fake that extends the one in the existing suite; it also models grants, one-off errors and dropped connections. Findings that could not be reproduced this way would have been dropped; none were.
3. **Grading.** Severity is graded for a production scheduled job using the rubric below.

Independence note. Claude Code (an AI coding assistant) wrote this loader earlier on 2026-11-13, as a refactor of the original `scripts/ingest_legacy_data.py`. The same assistant then ran the review as a separate pass. Readers should weigh the findings with that in mind; the failing tests are the evidence and can be checked independently.

SEVERITY RUBRIC

Severity	Meaning	Gate
CRITICAL	Breaks a dependent system or exposes or loses data in normal operation.	Must fix before GO
HIGH	Under a realistic fault, leaves wrong state or hides the failure from operators.	Must fix before GO
MEDIUM	Wrong behaviour only under specific conditions, or a structural problem that raises operating cost.	Fix within the first iteration after GO
LOW	Hardening, consistency or performance at scale.	Backlog

3. Findings

Each finding gives its location, what goes wrong, the failing proof test with its output, and the recommended fix. Test names are prefixed with the finding ID, for example `test_F01_…`.

F-01 · Refresh drops grants, indexes and dependent views on the live table

CRITICAL OPEN · `legacy_ingest/loader.py:212–229` (`_publish_swap`)

What happens. Refresh publishes by renaming the live table to `__OLD`, renaming staging into its place and dropping `__OLD`. The table seen by readers is therefore a new object after every refresh. Anything attached to the old object is dropped with it: object-level `GRANT`s, indexes added later, triggers and extended properties.

Failure scenario. A DBA runs `GRANT SELECT ON dbo.TBL_SC_FLEET_HIST_RAW TO USR_FDE_RO` so the agent can read telemetry. The next scheduled refresh drops that grant, and the agent's queries fail with a permission error. The loader logs `SUCCEEDED`. If `VW_ACTIVE_FLEET` is created `WITH SCHEMABINDING`, the rename fails instead and every refresh ends `FAILED`.

PROOF · `test_F01_refresh_preserves_permissions_on_live_table`

```
AssertionError: None != {'SELECT TO USR_FDE_RO'} : refresh dropped the GRANT the agent's
read-only login depends on
```

Recommended fix. Keep the live table as the same object and switch the data in instead. In one transaction run `TRUNCATE TABLE live; ALTER TABLE __STG SWITCH TO live;`. This is a metadata-only, effectively instant operation, and it requires the staging table to have identical columns, constraints and filegroup. It also removes the constraint-renaming step entirely. Add an integration test that creates a grant and a schema-bound view before the refresh.

F-02 · Failure handler crashes on a dead connection; audit row stuck at RUNNING

HIGH OPEN · `legacy_ingest/loader.py:165–171`

What happens. When a load fails, the handler calls `conn.rollback()` without any protection. If the connection is already gone, that call raises. The real error is replaced by the rollback error, the staging table is left behind, and the audit row is never updated. A Ctrl-C or `SIGTERM` during the load skips the handler in the same way.

Failure scenario. The link to the EC2-hosted SQL Server drops during the bulk insert (SQLSTATE `08S01`). The log shows only the rollback failure, and `TBL_SC_INGEST_AUDIT` shows a run that is `RUNNING` forever, so monitoring can't tell a hung load from a dead one.

PROOF · `test_F02_dead_connection_during_failure_handling_still_returns_failed_report`

```
RuntimeError: 08S01 communication link failure
During handling of the above exception, another exception occurred:
RuntimeError: 08S01 rollback on dead connection
AssertionError: run() raised from the failure handler instead of returning FAILED: 08S01
rollback on dead connection
```

Recommended fix. Run the rollback, staging cleanup and audit update independently, so that one failing doesn't skip the others. If the connection is dead, finish the audit row on a new connection. At the start of each run, while holding the lock, mark any leftover `RUNNING` rows for the table as `ABANDONED`. The lock guarantees no other run is active, so such rows are always stale.

F-03 · Deadlocks and dropped connections during publish are never retried

MEDIUM OPEN · `legacy_ingest/db.py:21, 53`

What happens. `TRANSIENT_SQLSTATES` includes `40001` (deadlock victim), which suggests retries. But the retry decorator wraps only `pyodbc.connect`. A deadlock during the `MERGE`, the swap or the bulk insert fails the run immediately.

Failure scenario. An upsert's `MERGE … WITH (HOLDLOCK)` deadlocks with a reader of `VW_ACTIVE_FLEET`, and SQL Server chooses the loader as the victim. The scheduled run exits 1 and pages the on-call engineer, even though simply running again would have succeeded.

PROOF · `test_F03_deadlock_victim_during_publish_is_retried`

```
AssertionError: 'FAILED' != 'SUCCEEDED' : deadlock not retried: Exception: ('40001',
'deadlock victim')
```

Recommended fix. Retry the whole publish transaction (rolling back between attempts) a bounded number of times on `40001` or error 1205. This is safe because staging is unchanged until the commit. Otherwise, remove `40001` from the constant so it no longer implies retries that don't happen.

F-04 · Upsert skips the load when the live table has been emptied

MEDIUM OPEN · `legacy_ingest/loader.py:177–187` (`_unchanged`)

What happens. In refresh mode, the skip-if-unchanged check also compares the live table's row count with the last successful load. In upsert mode it only checks that the table exists.

Failure scenario. After a successful `--mode upsert`, someone truncates the table, for example while cleaning up after an incident. The next scheduled upsert with the same CSV is `SKIPPED` and exits 0, so the agent sees no fleet history until someone thinks to pass `--force`.

PROOF · `test_F04_upsert_does_not_skip_when_live_table_was_emptied`

```
AssertionError: 'SKIPPED' == 'SKIPPED' : upsert skipped although the live table is empty
```

Recommended fix. Apply the row-count check in upsert mode as well. For upsert the condition is that the live table holds at least the number of rows last loaded. Alternatively, never skip in upsert mode; a `MERGE` that changes nothing is cheap.

F-05 · The audit hash can describe a different file from the one loaded

MEDIUM OPEN · `legacy_ingest/loader.py:140`, `extract.py:21–33`

What happens. The SHA-256 that drives the skip-if-unchanged check is computed in one pass over the file before the lock is taken. The rows are then parsed in a later, separate pass. If the file changes in between, the audit row records the old hash next to the new data.

Failure scenario. A re-download replaces the CSV while a run is starting. The table now holds version B, but the audit says version A was loaded. A later run with file A is then wrongly `SKIPPED`, and the table keeps B's data.

PROOF · `test_F05_audit_hash_matches_the_bytes_actually_loaded`

```
AssertionError: '24297cff…834a2' != '4c81005d…74f2b' : audit recorded the hash of a file
version that was not loaded
```

Recommended fix. Hash the bytes while they are being parsed: give `read_csv` a file object that updates the digest as it is read. Then take the skip decision either from a snapshot copy of the file or before parsing, using the same handle. This also fixes part of F-11.

F-06 · A CSV missing a column exits 1 (load failure) instead of 2 (config error)

MEDIUM OPEN · `legacy_ingest/loader.py:136–171` , `extract.py:14`

What happens. The header check runs inside the load's error handling. The `ConfigError` it raises is caught as a generic failure: the loader has already written an audit row and created the staging table, and it then exits 1. The CLI documents exit 2 for configuration errors, and `Loader.run` promises they surface "before any work". `--dry-run` on the same file correctly exits 2.

Failure scenario. The upstream dataset renames `iot_temperature` . The scheduler sees exit 1 ("data problem, retry later") instead of exit 2 ("broken configuration, needs a human"), so it retries the same failure every run.

PROOF · `test_F06_csv_missing_column_is_config_error_exit_2`

```
AssertionError: 1 != 2 : missing CSV column reported as a load failure (exit 1)
```

Recommended fix. Call `check_header` in `Loader.run` before connecting, alongside the file-exists check. If any `ConfigError` is still raised during the load, re-raise it as a configuration error after cleanup rather than turning it into a `FAILED` report.

F-07 · The loader cannot be deployed without the `rag/` package

MEDIUM OPEN · `legacy_ingest/config.py:16` , `cli.py:21` , `db.py:15`

What happens. The loader imports `PROJECT_ROOT` and `configure_logging` from `rag.config` , and its retry helper from `rag.core.resilience` . The refactor also changed `rag/core/resilience.py` to add a `predicate` parameter. Two unrelated services are now linked: the data-tier batch job and the SOP retrieval platform.

Failure scenario. A change to `rag/` 's settings or retry policy, such as a new required Pinecone setting, breaks or alters the legacy loader. A container built for the loader must also carry `rag/` and its configuration. Reviews of either package now pull in the other, which is exactly how the two reviews got mixed up.

PROOF · `test_F07_legacy_ingest_does_not_depend_on_rag`

```
AssertionError: Lists differ: ['cli.py: rag.config', 'config.py: rag.config', 'db.py:
rag.core.resilience'] != []
```

Recommended fix. Move the shared helpers (`PROJECT_ROOT` , `configure_logging` , the retry decorator) into a neutral `common/` package that both services import. Alternatively, copy the three small helpers into `legacy_ingest/` . Either way, revert or move the change to `rag/core/resilience.py` .

F-08 · A failed run does not report the reject file it already wrote

LOW OPEN · `legacy_ingest/loader.py:100–101`

What happens. `report.reject_file` is set only after every chunk has been read. If the load fails partway through, rejected rows are already on disk but the report says there are none, so the CLI's "Rejected rows written to …" warning never appears.

PROOF · `test_F08_failed_run_still_reports_written_reject_file`

```
AssertionError: None != '<tmp>/rejects/<run_id>.csv' : FAILED report hides the reject file
```

Recommended fix. Set `reject_file` as soon as `RejectSink` writes its first row.

F-09 · `--chunk-rows 0` is silently ignored

LOW OPEN · `legacy_ingest/cli.py:43`

What happens. `if args.chunk_rows:` treats 0 as "flag not given", so the run uses the default chunk size and exits 0. `--chunk-rows -1` correctly exits 2, and `--max-reject-ratio` on the line before already uses `is not None`.

PROOF · `test_F09_chunk_rows_zero_is_rejected`

```
AssertionError: 0 != 2 : --chunk-rows 0 silently ignored
```

Recommended fix. Use `if args.chunk_rows is not None:`.

F-10 · The identifier check accepts names ending in a newline

LOW OPEN · `legacy_ingest/schema.py:17`

What happens. The table-name check uses `re.match` with a pattern ending in `$`, and `$` also matches just before a trailing newline. A `TARGET_TABLE` value ending in `\n`, for example from a careless `.env` edit, passes. The loader then silently creates and publishes a differently named table that the agent's view never reads. The check still blocks SQL injection; this is a correctness problem, not a security one.

PROOF · `test_F10_identifier_with_trailing_newline_is_rejected`

```
AssertionError: ValueError not raised : quote_ident accepted 'TBL\n'
```

Recommended fix. Use `_IDENTIFIER.fullmatch(name)`, or end the pattern with `\Z`.

F-11 · The source file is read three times, and the duplicate check slows quadratically

LOW OPEN · `legacy_ingest/extract.py:14–33`, `validate.py:48`

What happens. Each load reads the CSV three times: once to hash it, once to check the header and once to parse it. Separately, `candidates.isin(seen_keys)` rebuilds a lookup table from every key seen so far on each chunk, so duplicate checking costs O(rows × chunks). The current 32k-row file still loads in about 3 s, but a multi-million-row history would spend most of its time on this.

PROOF · `test_F11_source_csv_is_read_in_a_single_pass`

```
AssertionError: 3 != 1 : source file read 3 times per load
```

Recommended fix. Read the file once: hash it while parsing (shared with F-05), and check the header against the first chunk's columns. For duplicates, test membership against the set directly with `candidates.map(seen_keys.__contains__)`, or keep a sorted key index, so each chunk costs time proportional to its own size.

4. Strengths Observed

The review also checked what the loader gets right. These properties were verified by tests or by the live run, and they should be kept through the fixes:

- **Readers never see a partial load.** Staging, reconciliation and a single-transaction publish mean a failure at any step leaves the live table untouched. The rollback path is covered by tests, and the swap was verified against SQL Server 2022.
- **Explicit, typed schema.** `TS_UTC DATETIME2(0)` is the primary key, measures are `FLOAT NOT NULL`, and `RISK_CLS_TXT` and `SYS_INGEST_FLAG` are protected by `CHECK` constraints. This replaces the original pandas-inferred table, which stored timestamps as text and had no key. Column names are unchanged for downstream users.
- **Data quality controls.** Nine row-level rules, duplicate detection across chunks, a quarantine CSV that records the reason for each rejection, and a configurable reject-ratio threshold that stops the run before anything is published.
- **Operability.** Every run is written to an audit table. Unchanged sources are skipped, with a guard for refresh mode against out-of-band edits. A session-scoped application lock blocks concurrent runs. Exit codes are documented (0/1/2/3), and `--dry-run` validates the CSV without needing the database or the ODBC driver.
- **Safe SQL construction.** Identifiers from configuration are checked against an allow-list and bracket-quoted, all values are passed as parameters, the password is held in a `SecretStr` that never appears in logs, and ODBC values are brace-escaped.
- **Speed.** Only the 9 needed columns are parsed, memory use is bounded by chunk size, and inserts are batched with `fast_executemany`. The live run loaded 32,065 rows over the network in 3.1 s; validating the full file in dry-run mode takes 0.15 s.
- **Tests.** 25 tests in `tests/test_legacy_ingest.py`: 24 offline, plus an integration test (opt-in with `LEGACY_DB_IT=1`) that runs refresh, skip and upsert against the real server and cleans up after itself.

5. Remediation Plan & Exit Criteria

ID	Fix	Effort	Gate
F-01	Publish by `TRUNCATE` + `ALTER TABLE … SWITCH`, drop constraint renaming, add a grant / schema-bound view integration test	M	**Before GO**
F-02	Run rollback, cleanup and audit update independently; finish audit on a fresh connection; mark stale `RUNNING` rows abandoned	S–M	**Before GO**
F-03	Bounded retry of the publish transaction on `40001` / 1205	S	First iteration after GO
F-04	Row-count check in upsert mode, or never skip upsert	S	First iteration after GO
F-05	Hash while parsing, one file handle	S	First iteration after GO
F-06	Check the header before connecting; keep exit 2 for config errors	S	First iteration after GO
F-07	Extract shared helpers into `common/`; revert or move the `rag/` change	S	First iteration after GO
F-08 – F-11	Report the reject file early; `is not None`; `fullmatch`; single-pass read and linear duplicate check	S each	Backlog

Effort: S = under half a day, M = about one day, including tests.

EXIT CRITERIA FOR GO

1. `test_F01_…` and `test_F02_…` pass and have been moved into `tests/test_legacy_ingest.py`.
2. The full suite is green: `venv/bin/python -m unittest discover -s tests`.
3. The integration test passes against a non-production SQL Server that has a `GRANT` on the table and a `VW_ACTIVE_FLEET` created `WITH SCHEMABINDING`. After a refresh, `USR_FDE_RO` can still query the view.
4. A connection-drop drill, where the network is cut during the bulk insert, leaves no `RUNNING` audit row and no `__STG` table after the next run.

Appendix A. How to Reproduce

From the repository root, with the project venv:

```
# 1. Reproduce all open findings (expected today: Ran 11 tests … FAILED (failures=11))
venv/bin/python -m unittest discover -s tests/open_findings -v

# 2. Main suite stays green (open-findings folder has no __init__.py, so it is not discovered)
venv/bin/python -m unittest discover -s tests

# 3. Optional live check against the SQL Server in .env (creates and removes
TBL_SC_FLEET_HIST_IT)
LEGACY_DB_IT=1 venv/bin/python -m unittest tests.test_legacy_ingest.IntegrationTests -v

# 4. Validate the source CSV without touching the database
venv/bin/python scripts/ingest_legacy_data.py --dry-run
```

Finding	**Test (in `tests/open_findings/test_legacy_open_findings.py`)**
F-01	`test_F01_refresh_preserves_permissions_on_live_table`
F-02	`test_F02_dead_connection_during_failure_handling_still_returns_failed_report`
F-03	`test_F03_deadlock_victim_during_publish_is_retried`
F-04	`test_F04_upsert_does_not_skip_when_live_table_was_emptied`
F-05	`test_F05_audit_hash_matches_the_bytes_actually_loaded`
F-06	`test_F06_csv_missing_column_is_config_error_exit_2`
F-07	`test_F07_legacy_ingest_does_not_depend_on_rag`
F-08	`test_F08_failed_run_still_reports_written_reject_file`
F-09	`test_F09_chunk_rows_zero_is_rejected`
F-10	`test_F10_identifier_with_trailing_newline_is_rejected`
F-11	`test_F11_source_csv_is_read_in_a_single_pass`

Environment used: Python 3.12.13, pandas 3.0.5, pyodbc 5.3.0, ODBC Driver 18 for SQL Server, SQL Server 2022 (Docker on EC2). When a finding is fixed, its test should start passing; move it into the main suite and mark the finding CLOSED in the next version of this report.